

DEVSPHERE

INTERNSHIP PROGRAM

WEEK 5 TASK REPORT

Artificial Intelligence (AI) & Machine Learning (ML)

Intern	Saud Masood
Program	DevSphere Internship Program
Domain	Artificial Intelligence & Machine Learning
Week	05
Task Areas	AI Model Implementation + Model Evaluation & Tuning
Deadline	8 October 2026

Internship Submission Report

1. Executive Summary

This report presents the completed Week 5 Artificial Intelligence and Machine Learning internship work assigned by DevSphere. The AI task develops an AI-based intelligent student performance prediction system. The ML task focuses on model evaluation and tuning by training multiple classification models, comparing evaluation metrics, and optimizing a Random Forest model through hyperparameter search.

2. Official Week 5 Requirements

Area	Requirements	Output
AI	Train AI/ML model Data preprocessing Prediction system Accuracy evaluation User input handling Result visualization	Working AI-based intelligent system
ML	Train multiple models Compare performance Apply model tuning Use evaluation metrics Show graphs/charts for results	Optimized ML model with evaluation report

3. AI Task — AI Model Implementation

Topic: AI Model Implementation

Project: Student Performance Prediction System

3.1 Objective

The system performs an end-to-end prediction workflow: loading the dataset, preprocessing numerical data, training a Logistic Regression model, evaluating test accuracy, accepting new student information, predicting Pass/Fail, and visualizing prediction probabilities.

3.2 AI Workflow

Dataset → Data Preprocessing → Train/Test Split → Feature Scaling → Model Training → Accuracy Evaluation → User Input → Prediction → Result Visualization

3.3 AI Dataset

The AI dataset contains StudyHours, Attendance, PreviousScore, and Passed. The Passed column is the target variable, where 0 represents Fail and 1 represents Pass.

3.4 AI Implementation Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv("student_performance.csv")

numeric_columns = df.select_dtypes(include="number").columns
for column in numeric_columns:
    df[column] = df[column].fillna(df[column].mean())

X = df[["StudyHours", "Attendance", "PreviousScore"]]
y = df["Passed"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")

study_hours = float(input("Study Hours: "))
attendance = float(input("Attendance Percentage: "))
previous_score = float(input("Previous Exam Score: "))

user_data = pd.DataFrame({
    "StudyHours": [study_hours],
    "Attendance": [attendance],
    "PreviousScore": [previous_score]
})

user_data_scaled = scaler.transform(user_data)
prediction = model.predict(user_data_scaled)
probability = model.predict_proba(user_data_scaled)

result = "PASS" if prediction[0] == 1 else "FAIL"
```

```

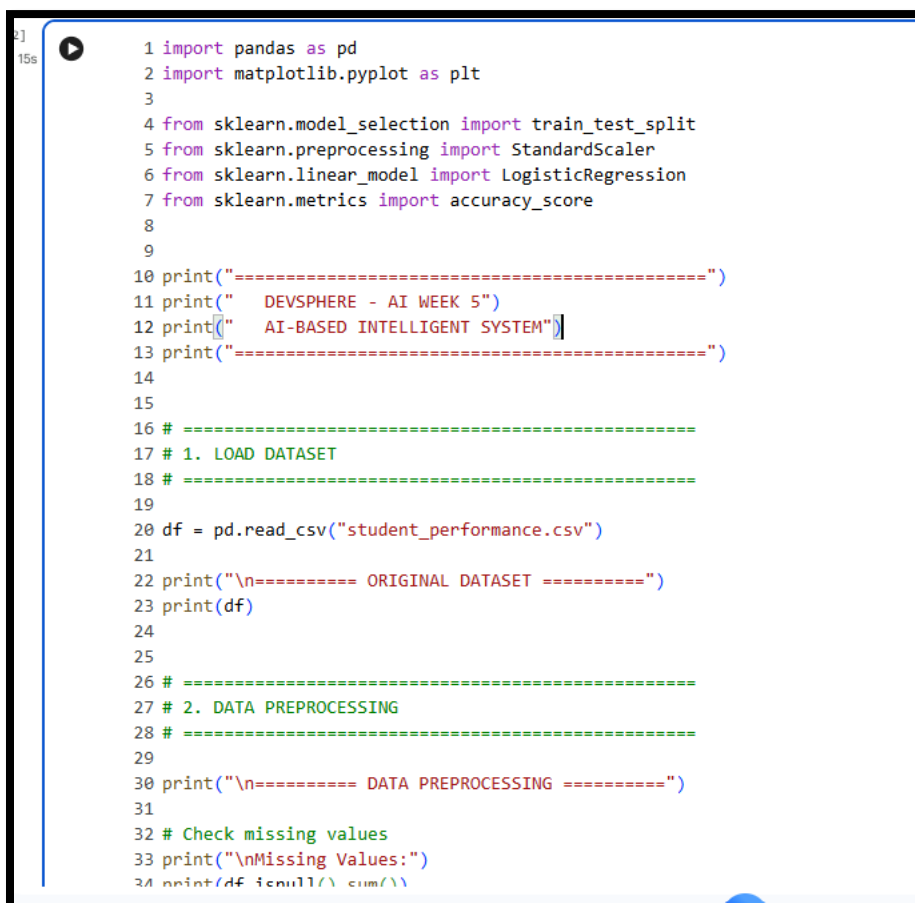
print("Prediction:", result)
print(f"Pass Probability: {probability[0][1] * 100:.2f}%")
print(f"Fail Probability: {probability[0][0] * 100:.2f}%")

labels = ["Pass Probability", "Fail Probability"]
values = [probability[0][1] * 100, probability[0][0] * 100]

plt.figure(figsize=(8, 5))
plt.bar(labels, values)
plt.title("AI Student Prediction Result")
plt.ylabel("Probability (%)")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()

```

AI CODE EXECUTION SCREENSHOT



```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import accuracy_score
8
9
10 print("=====")
11 print("  DEVSPHERE - AI WEEK 5")
12 print("  AI-BASED INTELLIGENT SYSTEM")
13 print("=====")
14
15
16 # =====
17 # 1. LOAD DATASET
18 # =====
19
20 df = pd.read_csv("student_performance.csv")
21
22 print("\n===== ORIGINAL DATASET =====")
23 print(df)
24
25
26 # =====
27 # 2. DATA PREPROCESSING
28 # =====
29
30 print("\n===== DATA PREPROCESSING =====")
31
32 # Check missing values
33 print("\nMissing Values:")
34 print(df.isnull().sum())

```

[2]
✓ 15s



```
32 # Check missing values
33 print("\nMissing Values:")
34 print(df.isnull().sum())
35
36 # Fill missing numerical values with mean
37 numeric_columns = df.select_dtypes(include="number").columns
38
39 for column in numeric_columns:
40     df[column] = df[column].fillna(df[column].mean())
41
42 print("\nMissing values after preprocessing:")
43 print(df.isnull().sum())
44
45 print("\nData preprocessing completed successfully!")
46
47
48 # =====
49 # 3. SELECT FEATURES AND TARGET
50 # =====
51
52 X = df[["StudyHours", "Attendance", "PreviousScore"]]
53 y = df["Passed"]
54
55
56 # =====
57 # 4. TRAIN / TEST SPLIT
58 # =====
59
60 X_train, X_test, y_train, y_test = train_test_split(
61     X,
62     y,
63     test_size=0.25,
64     random_state=42,
65     stratify=y
```

[2]
✓ 15s



```
66 )
67
68
69 # =====
70 # 5. FEATURE SCALING
71 # =====
72
73 scaler = StandardScaler()
74
75 X_train_scaled = scaler.fit_transform(X_train)
76 X_test_scaled = scaler.transform(X_test)
77
78
79 # =====
80 # 6. TRAIN AI/ML MODEL
81 # =====
82
83 model = LogisticRegression()
84
85 model.fit(X_train_scaled, y_train)
86
87 print("\n===== MODEL TRAINING =====")
88 print("AI/ML model trained successfully!")
89
90
91 # =====
92 # 7. ACCURACY EVALUATION
93 # =====
94
95 y_pred = model.predict(X_test_scaled)
96
97 accuracy = accuracy_score(y_test, y_pred)
98
99 print("\n===== MODEL EVALUATION =====")
100 print("Actual Values: ", list(y_test))
```



```

100 print("Actual Values: ", list(y_test))
101 print("Predicted Values:", list(y_pred))
102
103 print(f"\nModel Accuracy: {accuracy * 100:.2f}%")
104
105
106 # =====
107 # 8. USER INPUT HANDLING
108 # =====
109
110 print("\n=====")
111 print("      STUDENT PREDICTION SYSTEM")
112 print("=====")
113
114 print("\nEnter student information:")
115
116 study_hours = float(
117 | | input("Study Hours: ")
118 )
119
120 attendance = float(
121 | | input("Attendance Percentage: ")
122 )
123
124 previous_score = float(
125 | | input("Previous Exam Score: ")
126 )
127
128
129 # =====
130 # 9. CREATE USER INPUT DATA
131 # =====
132
133 user_data = pd.DataFrame({
134 | | "Study_Hours": study_hours,

```

```

132
133 user_data = pd.DataFrame({
134     "StudyHours": [study_hours],
135     "Attendance": [attendance],
136     "PreviousScore": [previous_score]
137 })
138
139
140 # Apply the same preprocessing/scaling
141 user_data_scaled = scaler.transform(user_data)
142
143
144 # =====
145 # 10. MAKE PREDICTION
146 # =====
147
148 prediction = model.predict(user_data_scaled)
149
150 prediction_probability = model.predict_proba(
151     user_data_scaled
152 )
153
154 if prediction[0] == 1:
155     result = "PASS"
156 else:
157     result = "FAIL"
158
159
160 # =====
161 # 11. DISPLAY RESULT
162 # =====
163
164 print("\n=====")
165 print("          AI PREDICTION RESULT")
166 print("=====")

```

```
166 print(
167
168 print(f"Study Hours: {study_hours}")
169 print(f"Attendance: {attendance}%")
170 print(f"Previous Score: {previous_score}")
171
172 print("\nPrediction:", result)
173
174 print(
175     f"Pass Probability: "
176     f"{prediction_probability[0][1] * 100:.2f}%"
177 )
178
179 print(
180     f"Fail Probability: "
181     f"{prediction_probability[0][0] * 100:.2f}%"
182 )
183
184
185 # =====
186 # 12. RESULT VISUALIZATION
187 # =====
188
189 print("\nGenerating result visualization...")
190
191 labels = ["Pass Probability", "Fail Probability"]
192
193 probabilities = [
194     prediction_probability[0][1] * 100,
195     prediction_probability[0][0] * 100
196 ]
197
198 plt.figure(figsize=(8, 5))
199
```



```

197
198 plt.figure(figsize=(8, 5))
199 |
200 bars = plt.bar(
201 |     labels,
202 |     probabilities
203 | )
204
205 plt.title("AI Student Prediction Result")
206 plt.xlabel("Prediction")
207 plt.ylabel("Probability (%)")
208
209 plt.ylim(0, 100)
210
211 for bar, value in zip(bars, probabilities):
212 |     plt.text(
213 |         bar.get_x() + bar.get_width() / 2,
214 |         value + 2,
215 |         f"{value:.2f}%",
216 |         ha="center"
217 |     )
218
219 plt.tight_layout()
220
221 plt.show()
222
223
224 # =====
225 # FINAL MESSAGE
226 # =====
227
228 print("\n=====")
229 print("    AI-BASED INTELLIGENT SYSTEM COMPLETED")
230 print("=====")

```

AI OUTPUT + VISUALIZATION SCREENSHOT

```

=====
...  DEVSPHERE - AI WEEK 5
      AI-BASED INTELLIGENT SYSTEM
=====

===== ORIGINAL DATASET =====
      StudyHours  Attendance  PreviousScore  Passed
0              1           50             45      0
1              2           55             50      0
2              2           60             52      0
3              3           65             55      0
4              3           68             58      0
5              4           70             62      1
6              5           72             65      1
7              5           75             68      1
8              6           78             70      1
9              7           80             75      1
10             8           85             80      1
11             9           88             85      1
12             9           90             88      1
13            10           95             92      1
14            10           98             95      1

===== DATA PREPROCESSING =====

Missing Values:
StudyHours      0
Attendance      0
PreviousScore   0
Passed          0
dtype: int64

Missing values after preprocessing:
StudyHours      0
Attendance      0
PreviousScore   0
Passed          0

```

```

dtype: int64

Data preprocessing completed successfully!

===== MODEL TRAINING =====
AI/ML model trained successfully!

===== MODEL EVALUATION =====
Actual Values:  [1, 1, 0, 1]
Predicted Values: [np.int64(1), np.int64(1), np.int64(0), np.int64(1)]

Model Accuracy: 100.00%

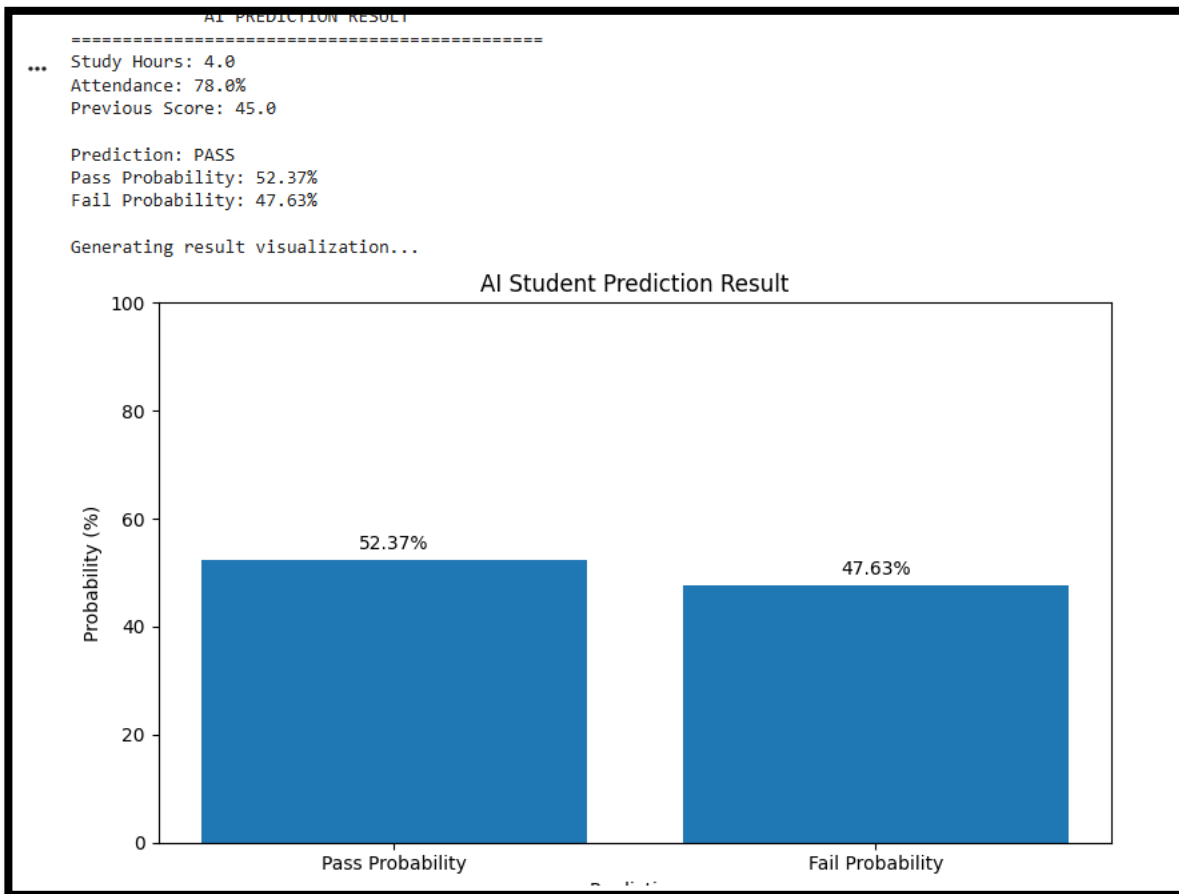
=====
      STUDENT PREDICTION SYSTEM
=====

Enter student information:
Study Hours: 4
Attendance Percentage: 78
Previous Exam Score: 45

=====
      AI PREDICTION RESULT
=====
Study Hours: 4.0
Attendance: 78.0%
Previous Score: 45.0

Prediction: PASS
Pass Probability: 52.37%
Fail Probability: 47.63%

```



3.5 AI Implementation Explanation

- Data preprocessing: numerical columns are checked and missing numerical values are replaced with the corresponding column mean.
- Model preparation: StudyHours, Attendance, and PreviousScore are used as input features, while Passed is the target.
- Training: the data is split into training and testing subsets, standardized using StandardScaler, and used to train Logistic Regression.
- Evaluation: predictions on the test set are compared with the actual values using accuracy_score.
- User interaction: the system accepts Study Hours, Attendance, and Previous Exam Score from the user.
- Prediction and visualization: the trained model predicts Pass/Fail and displays pass/fail probabilities in a Matplotlib bar chart.

4. ML Task — Model Evaluation + Tuning

Topic: Model Evaluation + Tuning

Task: Optimize Machine Learning Model

4.1 Objective

The ML implementation trains multiple classification models, compares their performance using Accuracy, Precision, Recall, and F1 Score, applies GridSearchCV to tune a Random Forest classifier, and presents the evaluation through charts and a confusion matrix.

4.2 Models

- Logistic Regression
- Decision Tree
- Random Forest

4.3 Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1 Score

4.4 Hyperparameter Tuning

GridSearchCV searches combinations of Random Forest hyperparameters including `n_estimators`, `max_depth`, and `min_samples_split`. Three-fold cross-validation is used with accuracy as the scoring metric.

4.5 ML Dataset

ML DATASET / CSV use Attached in Email submission

4.6 ML Implementation Code

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

df = pd.read_csv("student_classification_week5.csv")

numeric_columns = df.select_dtypes(include="number").columns
for column in numeric_columns:
    df[column] = df[column].fillna(df[column].mean())

X = df[["StudyHours", "Attendance"]]
y = df["Passed"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```

models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression())
    ]),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    )
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, predictions, zero_division=0
        )
    })

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

param_grid = {
    "n_estimators": [50, 100, 150],
    "max_depth": [None, 2, 3, 5],
    "min_samples_split": [2, 3, 4]
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=3,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

optimized_model = grid_search.best_estimator_
optimized_predictions = optimized_model.predict(X_test)

accuracy = accuracy_score(y_test, optimized_predictions)
precision = precision_score(
    y_test, optimized_predictions, zero_division=0
)
recall = recall_score(
    y_test, optimized_predictions, zero_division=0
)
f1 = f1_score(
    y_test, optimized_predictions, zero_division=0
)

print("Best Parameters:", grid_search.best_params_)
print(f"Accuracy: {accuracy * 100:.2f}%")
print(f"Precision: {precision * 100:.2f}%")
print(f"Recall: {recall * 100:.2f}%")
print(f"F1 Score: {f1 * 100:.2f}%")

print(classification_report(
    y_test,
    optimized_predictions,
    target_names=["Fail", "Pass"],
    zero_division=0
))

cm = confusion_matrix(y_test, optimized_predictions)

plt.figure(figsize=(10, 6))
x = range(len(results_df))

plt.bar(
    [i - 0.3 for i in x],
    results_df["Accuracy"],

```

```

        width=0.2,
        label="Accuracy"
    )
    plt.bar(
        [i - 0.1 for i in x],
        results_df["Precision"],
        width=0.2,
        label="Precision"
    )
    plt.bar(
        [i + 0.1 for i in x],
        results_df["Recall"],
        width=0.2,
        label="Recall"
    )
    plt.bar(
        [i + 0.3 for i in x],
        results_df["F1 Score"],
        width=0.2,
        label="F1 Score"
    )

plt.xticks(x, results_df["Model"])
plt.ylabel("Score")
plt.xlabel("Machine Learning Model")
plt.title("Machine Learning Model Performance Comparison")
plt.ylim(0, 1.1)
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(6, 5))
plt.imshow(cm)
plt.title("Optimized Model Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.xticks([0, 1], ["Fail", "Pass"])
plt.yticks([0, 1], ["Fail", "Pass"])

for i in range(len(cm)):
    for j in range(len(cm[i])):
        plt.text(
            j, i, cm[i][j],
            ha="center",
            va="center"
        )

plt.colorbar()
plt.tight_layout()
plt.show()

metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]
values = [accuracy, precision, recall, f1]

plt.figure(figsize=(8, 5))
bars = plt.bar(metrics, values)
plt.title("Optimized Random Forest Model Metrics")
plt.ylabel("Score")
plt.ylim(0, 1.1)

for bar, value in zip(bars, values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.02,
        f"{value * 100:.2f}%",
        ha="center"
    )

plt.tight_layout()
plt.show()

```

ML CODE EXECUTION SCREENSHOT

```
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split, GridSearchCV
6
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.tree import DecisionTreeClassifier
9 from sklearn.ensemble import RandomForestClassifier
10
11 from sklearn.metrics import (
12     accuracy_score,
13     precision_score,
14     recall_score,
15     f1_score,
16     classification_report,
17     confusion_matrix,
18     ConfusionMatrixDisplay
19 )
20
21
22 # =====
23 # 1. PROGRAM HEADER
24 # =====
25
26 print("=" * 60)
27 print("          DEVSPHERE - MACHINE LEARNING WEEK 5")
28 print("          MODEL EVALUATION + TUNING")
29 print("=" * 60)
30
31
32 # =====
33 # 2. LOAD DATASET
34 # =====
35
36 print("\n" + "=" * 60)
37 print("          DATASET")
38 print("=" * 60)
39
40 df = pd.read_csv("student_classification_week5.csv")
41
42 print("\nDataset loaded successfully!")
43
44 print("\nDataset:")
45 print(df)
46
47 print("\nDataset Information:")
48 print(df.info())
49
50 print("\nDataset Shape:")
51 print(df.shape)
```

```
54 # =====
55 # 3. DATA PREPROCESSING
56 # =====
57
58 print("\n" + "=" * 60)
59 print("          DATA PREPROCESSING")
60 print("=" * 60)
61
62 print("\nMissing Values:")
63 print(df.isnull().sum())
64
65 # Fill missing numerical values if any
66 numeric_columns = df.select_dtypes(include="number").columns
67
68 for column in numeric_columns:
69     df[column] = df[column].fillna(df[column].mean())
70
71 # Remove duplicate rows
72 df = df.drop_duplicates()
73
74 print("\nMissing Values After Preprocessing:")
75 print(df.isnull().sum())
76
77 print("\nData preprocessing completed successfully!")
78
79
80 # =====
81 # 4. DEFINE FEATURES AND TARGET
82 # =====
83
84 X = df[[
85     "StudyHours",
86     "Attendance"
87 ]]
88
```

```

94
95 print("\nFeatures:")
96 print(X)
97
98 print("\nTarget:")
99 print(y)
100
101
102 # =====
103 # 5. TRAIN-TEST SPLIT
104 # =====
105
106 X_train, X_test, y_train, y_test = train_test_split(
107     X,
108     y,
109     test_size=0.25,
110     random_state=42,
111     stratify=y
112 )
113
114 print("\n" + "=" * 60)
115 print("                                TRAIN TEST SPLIT")
116 print("=" * 60)
117
118 print("\nTraining samples:", len(X_train))
119 print("Testing samples:", len(X_test))
120
121
122 # =====
123 # 6. TRAIN MULTIPLE MACHINE LEARNING MODELS
124 # =====
125
126 print("\n" + "=" * 60)
127 print("                                TRAINING MULTIPLE MODELS")
128 print("=" * 60)

```



```
133 # -----
134
135 logistic_model = LogisticRegression(
136 |     random_state=42
137 )
138
139 logistic_model.fit(
140 |     X_train,
141 |     y_train
142 )
143
144 logistic_predictions = logistic_model.predict(
145 |     X_test
146 )
147
148
149 # -----
150 # Decision Tree
151 # -----
152
153 decision_tree_model = DecisionTreeClassifier(
154 |     random_state=42
155 )
156
157 decision_tree_model.fit(
158 |     X_train,
159 |     y_train
160 )
161
162 decision_tree_predictions = decision_tree_model.predict(
163 |     X_test
164 )
165
166
167 # -----
168 # Random Forest
169 # -----
170
171 random_forest_model = RandomForestClassifier(
172 |     random_state=42
173 )
174
175 random_forest_model.fit(
176 |     X_train,
177 |     y_train
178 )
179
180 random_forest_predictions = random_forest_model.predict(
181 |     X_test
182 )
```

```
177 | y_train
178 | )
179
180 random_forest_predictions = random_forest_model.predict(
181 | X_test
182 | )
183
184
185 print("\nAll three models trained successfully!")
186
187
188 # =====
189 # 7. EVALUATION FUNCTION
190 # =====
191
192 def evaluate_model(
193 | model_name,
194 | y_true,
195 | y_pred
196 | ):
197
198     accuracy = accuracy_score(
199 | y_true,
200 | y_pred
201 | )
202
203     precision = precision_score(
204 | y_true,
205 | y_pred,
206 | zero_division=0
207 | )
208
209     recall = recall_score(
210 | y_true,
211 | y_pred,
```

```
210     y_true,
211     y_pred,
212     zero_division=0
213 )
214
215 f1 = f1_score(
216     y_true,
217     y_pred,
218     zero_division=0
219 )
220
221 print("\n" + "-" * 60)
222 print(model_name)
223 print("-" * 60)
224
225 print(
226     f"Accuracy : {accuracy * 100:.2f}%"
227 )
228
229 print(
230     f"Precision: {precision * 100:.2f}%"
231 )
232
233 print(
234     f"Recall   : {recall * 100:.2f}%"
235 )
236
237 print(
238     f"F1 Score : {f1 * 100:.2f}%"
239 )
240
241 return {
242     "Model": model_name,
243     "Accuracy": accuracy,
244     "Precision": precision,
```

```

263         y_test,
264         logistic_predictions
265     )
266 )
267
268 results.append(
269     evaluate_model(
270         "Decision Tree",
271         y_test,
272         decision_tree_predictions
273     )
274 )
275
276 results.append(
277     evaluate_model(
278         "Random Forest",
279         y_test,
280         random_forest_predictions
281     )
282 )
283
284
285 # =====
286 # 9. MODEL COMPARISON TABLE
287 # =====
288
289 results_df = pd.DataFrame(results)
290
291 print("\n" + "=" * 60)
292 print("                MODEL COMPARISON")
293 print("=" * 60)
294
295 print("\n")
296 print(results_df.to_string(index=False))
297

```

```
302
303 print("\nGenerating model comparison chart...")
304
305 metrics = [
306     "Accuracy",
307     "Precision",
308     "Recall",
309     "F1 Score"
310 ]
311
312 x = range(len(results_df["Model"]))
313
314 plt.figure(figsize=(10, 6))
315
316 width = 0.2
317
318 for i, metric in enumerate(metrics):
319     values = results_df[metric]
320
321     positions = [
322         position + (i - 1.5) * width
323         for position in x
324     ]
325
326     plt.bar(
327         positions,
328         values,
329         width=width,
330         label=metric
331     )
332
333
334 plt.xticks(
335     list(x),
336     results_df["Model"]
```

```
355
356 plt.show()
357
358
359 # =====
360 # 11. HYPERPARAMETER TUNING
361 # =====
362
363 print("\n" + "=" * 60)
364 print("                                MODEL TUNING")
365 print("=" * 60)
366
367 print("\nTuning Random Forest model...")
368
369
370 param_grid = {
371
372     "n_estimators": [
373         50,
374         100,
375         150
376     ],
377
378     "max_depth": [
379         None,
380         2,
381         3,
382         5
383     ],
384
385     "min_samples_split": [
386         2,
387         3,
388         4
```

```
392
393 grid_search = GridSearchCV(
394     estimator=RandomForestClassifier(
395         random_state=42
396     ),
397     param_grid=param_grid,
398     cv=2,
399     scoring="accuracy",
400     n_jobs=-1
401 )
402
403 grid_search.fit(
404     X_train,
405     y_train
406 )
407
408 # IMPORTANT:
409 # Correct sklearn attributes use ONE underscore at the end.
410
411 print("\nBest Parameters:")
412
413 print(
414     grid_search.best_params_
415 )
416
417 print(
418     f"\nBest Cross-Validation Accuracy: "
```

```

421 | grid_search.best_params_
422 |
423 |
424 |
425 | print(
426 |     f"\nBest Cross-Validation Accuracy: "
427 |     f"{grid_search.best_score_ * 100:.2f}%"
428 | )
429 |
430 |
431 | # =====
432 | # 12. GET OPTIMIZED MODEL
433 | # =====
434 |
435 | optimized_model = grid_search.best_estimator_
436 |
437 | print("\nOptimized Random Forest model obtained successfully!")
438 |
439 |
440 | # =====
441 | # 13. EVALUATE OPTIMIZED MODEL
442 | # =====
443 |
444 | optimized_predictions = optimized_model.predict(
445 |     X_test
446 | )
447 |
448 |
449 | optimized_accuracy = accuracy_score(
450 |     y_test,
451 |     optimized_predictions
452 | )
453 |
454 | optimized_precision = precision_score(

```

```

462 |     optimized_predictions,
463 |     zero_division=0
464 | )
465 |
466 | optimized_f1 = f1_score(
467 |     y_test,
468 |     optimized_predictions,
469 |     zero_division=0
470 | )
471 |
472 |
473 | print("\n" + "=" * 60)
474 | print("          OPTIMIZED MODEL RESULTS")
475 | print("=" * 60)
476 |
477 | print(
478 |     f"\nAccuracy : {optimized_accuracy * 100:.2f}%"
479 | )
480 |
481 | print(
482 |     f"Precision: {optimized_precision * 100:.2f}%"
483 | )
484 |
485 | print(
486 |     f"Recall   : {optimized_recall * 100:.2f}%"
487 | )
488 |
489 | print(
490 |     f"F1 Score : {optimized_f1 * 100:.2f}%"
491 | )

```

```

509
510
511 # =====
512 # 15. CONFUSION MATRIX
513 # =====
514
515 print("\nGenerating confusion matrix..")
516
517 cm = confusion_matrix(
518     y_test,
519     optimized_predictions
520 )
521
522 print("\nConfusion Matrix:")
523 print(cm)
524
525
526 disp = ConfusionMatrixDisplay(
527     confusion_matrix=cm,
528     display_labels=["Fail", "Pass"]
529 )
530
531 disp.plot()
532
533 plt.title(
534     "Optimized Random Forest - Confusion Matrix"
535 )
536
537 plt.tight_layout()
538
539 plt.show()
540

```

```

545
546 print("\nGenerating optimized model metrics chart...")
547
548 optimized_metrics = {
549     "Accuracy":
550         optimized_accuracy,
551     "Precision":
552         optimized_precision,
553     "Recall":
554         optimized_recall,
555     "F1 Score":
556         optimized_f1
557 }
558
559
560
561
562
563
564 plt.figure(
565     figsize=(8, 5)
566 )
567
568 plt.bar(
569     optimized_metrics.keys(),
570     optimized_metrics.values()
571 )
572
573 plt.title(
574     "Optimized Random Forest Performance"
575 )
576
577 plt.xlabel(
578     "Evaluation Metric"
579 )

```

```

571 )
572
573 plt.title(
574 | "Optimized Random Forest Performance"
575 )
576
577 plt.xlabel(
578 | "Evaluation Metric"
579 )
580
581 plt.ylabel(
582 | "Score"
583 )
584
585 plt.ylim(
586 | 0,
587 | 1.1
588 )
589
590 plt.tight_layout()
591
592 plt.show()
593
594
595 # -----
596 # 17. FINAL OUTPUT
597 # -----
598
599 print("\n" + "=" * 60)
600 print("                FINAL RESULT")
601 print("=" * 60)
602
603 print("\nModels trained:")
604 print("1. Logistic Regression")
605 print("2. Decision Tree")

```

```

598
599 print("\n" + "=" * 60)
600 print("                FINAL RESULT")
601 print("=" * 60)
602
603 print("\nModels trained:")
604 print("1. Logistic Regression")
605 print("2. Decision Tree")
606 print("3. Random Forest")
607
608 print("\nEvaluation metrics used:")
609 print("1. Accuracy")
610 print("2. Precision")
611 print("3. Recall")
612 print("4. F1 Score")
613
614 print("\nModel tuning:")
615 print("Random Forest hyperparameters optimized using GridSearchCV.")
616
617 print(
618 | f"\nBest Cross-Validation Accuracy: "
619 | f"{grid_search.best_score_ * 100:.2f}%"
620 )
621
622 print(
623 | f"Optimized Test Accuracy: "
624 | f"{optimized_accuracy * 100:.2f}%"
625 )
626
627 print("\n" + "=" * 60)
628 print("                ML WEEK 5 TASK COMPLETED SUCCESSFULLY")
629 print("=" * 60)

```

ML MODEL COMPARISON / OUTPUT SCREENSHOT

```

... -----
      DEVSHERE - MACHINE LEARNING WEEK 5
      MODEL EVALUATION + TUNING
      -----

      DATASET
      -----

Dataset loaded successfully!

Dataset:
   StudyHours  Attendance  Passed
0           1           50       0
1           2           55       0
2           2           60       0
3           3           65       0
4           4           70       1
5           5           72       1
6           5           75       1
7           6           78       1
8           7           80       1
9           8           85       1
10          9           90       1
11         10           95       1

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype

```

```
11      10      99      1

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   StudyHours  12 non-null    int64
1   Attendance  12 non-null    int64
2   Passed      12 non-null    int64
dtypes: int64(3)
memory usage: 420.0 bytes
None

Dataset Shape:
(12, 3)
```

```
..          DATA PREPROCESSING
-----

Missing Values:
StudyHours    0
Attendance    0
Passed        0
dtype: int64

Missing Values After Preprocessing:
StudyHours    0
Attendance    0
Passed        0
dtype: int64

Data preprocessing completed successfully!
```

```

... =====
                FEATURES AND TARGET
=====

Features:
  StudyHours  Attendance
0           1          50
1           2          55
2           2          60
3           3          65
4           4          70
5           5          72
6           5          75
7           6          78
8           7          80
9           8          85
10          9          90
11         10          95

Target:
0      0
1      0
2      0
3      0
4      1
5      1
6      1
7      1
8      1
9      1
10     1

```

```

11      1
Name: Passed, dtype: int64

```

```

=====
                TRAIN TEST SPLIT
=====

```

```

Training samples: 9
Testing samples: 3

```

```

=====
                TRAINING MULTIPLE MODELS
=====

```

```

All three models trained successfully!

```

```

=====
                MODEL EVALUATION
=====

```

```

-----
Logistic Regression
-----

```

```

Accuracy : 100.00%
Precision: 100.00%
Recall   : 100.00%
F1 Score : 100.00%

```

```
...
-----
Decision Tree
-----
Accuracy : 100.00%
Precision: 100.00%
Recall   : 100.00%
F1 Score : 100.00%

-----

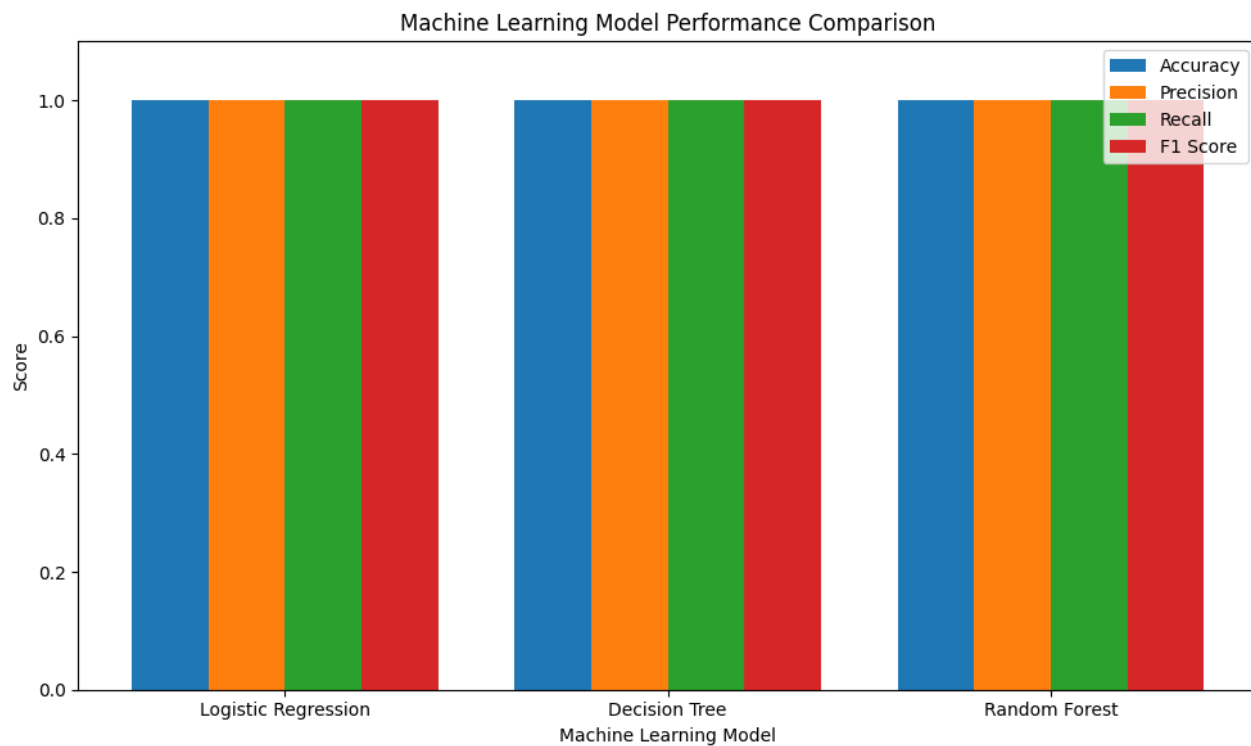
Random Forest
-----
Accuracy : 100.00%
Precision: 100.00%
Recall   : 100.00%
F1 Score : 100.00%

=====
MODEL COMPARISON
=====

      Model  Accuracy  Precision  Recall  F1 Score
Logistic Regression      1.0       1.0     1.0     1.0
      Decision Tree      1.0       1.0     1.0     1.0
      Random Forest      1.0       1.0     1.0     1.0

Generating model comparison chart...
```

... Generating model comparison chart...



```

=====
                        MODEL TUNING
=====

Tuning Random Forest model...

Best Parameters:
{'max_depth': None, 'min_samples_split': 2, 'n_estimators': 50}

Best Cross-Validation Accuracy: 77.50%

Optimized Random Forest model obtained successfully!

=====
                        OPTIMIZED MODEL RESULTS
=====

Accuracy : 100.00%
Precision: 100.00%
Recall   : 100.00%
F1 Score : 100.00%

```

```

Accuracy : 100.00%
Precision: 100.00%
Recall   : 100.00%
F1 Score : 100.00%

```

```

=====
                        CLASSIFICATION REPORT
=====

```

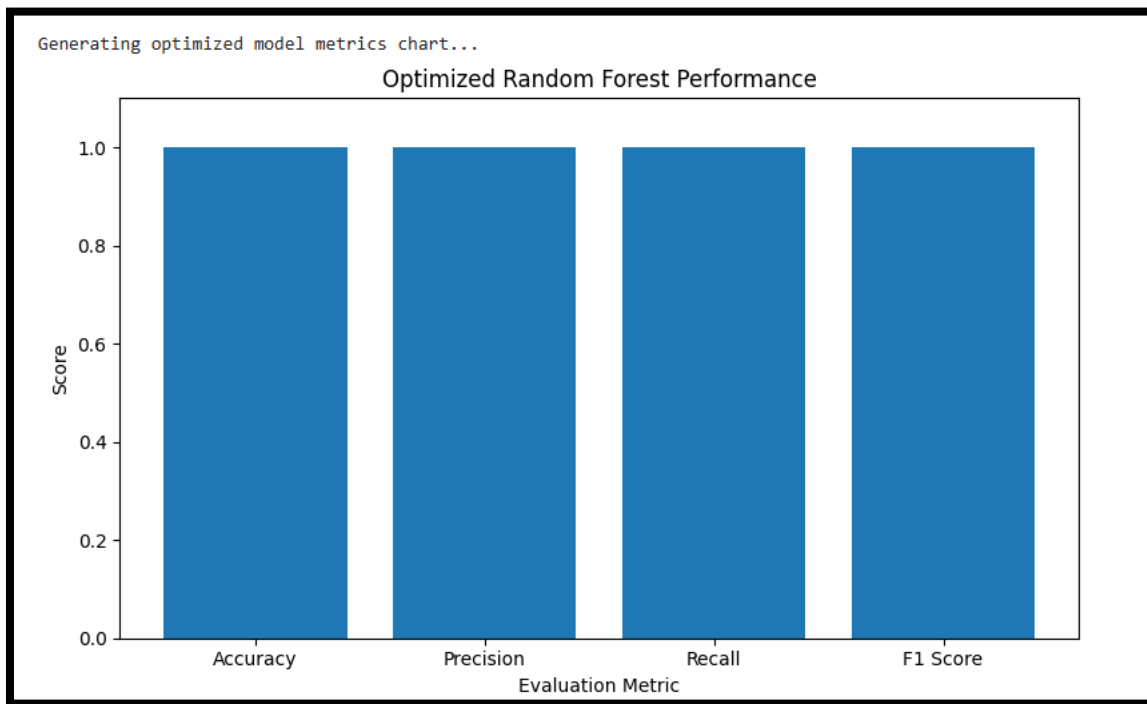
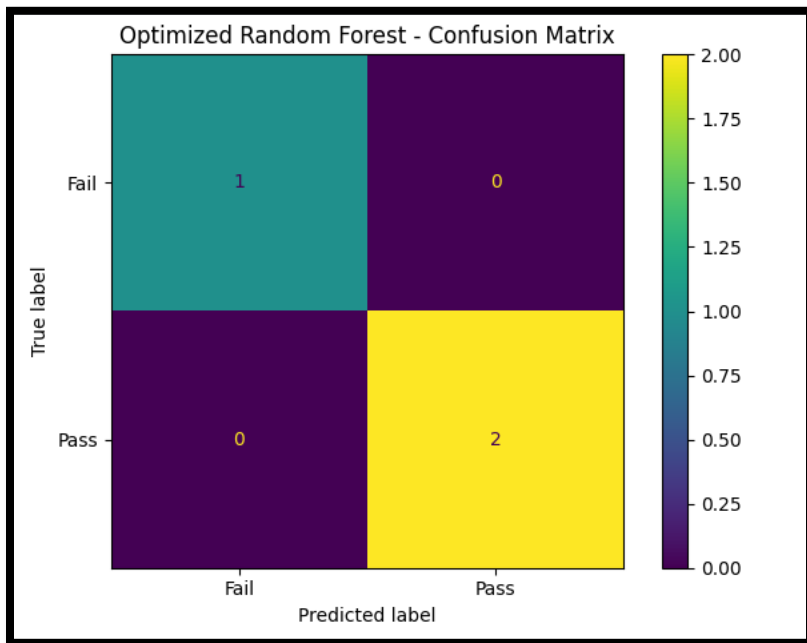
	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Generating confusion matrix...

```

Confusion Matrix:
[[1 0]
 [0 2]]

```



```
=====
                        FINAL RESULT
=====

Models trained:
1. Logistic Regression
2. Decision Tree
3. Random Forest

Evaluation metrics used:
1. Accuracy
2. Precision
3. Recall
4. F1 Score

Model tuning:
Random Forest hyperparameters optimized using GridSearchCV.

Best Cross-Validation Accuracy: 77.50%
Optimized Test Accuracy: 100.00%

=====
                ML WEEK 5 TASK COMPLETED SUCCESSFULLY
=====
```

4.7 ML Implementation Explanation

- Data preprocessing: the classification dataset is loaded and numerical missing values are handled using column means.
- Multiple models: Logistic Regression, Decision Tree, and Random Forest are trained using the same train/test split.
- Performance comparison: Accuracy, Precision, Recall, and F1 Score are calculated for every model and displayed in a comparison chart.
- Model tuning: GridSearchCV evaluates Random Forest hyperparameter combinations using three-fold cross-validation.
- Optimized evaluation: the best Random Forest configuration is evaluated on the held-out test data.

- Visualization: the implementation displays a model comparison chart, confusion matrix, and optimized-model metric chart.

5. Results & Discussion

The AI implementation produces a working student performance prediction workflow, while the ML implementation provides a structured model evaluation and tuning workflow. The actual metric values, best hyperparameters, predictions, and charts should be taken from the execution screenshots included in this report.

5.1 AI Result

The AI system trains a Logistic Regression model, reports test accuracy, accepts user input, produces a Pass/Fail prediction, and visualizes the prediction probabilities.

5.2 ML Result

The ML system compares three classification models and applies GridSearchCV tuning to Random Forest. The final evaluation includes Accuracy, Precision, Recall, F1 Score, classification report, confusion matrix, and visual comparisons.

FINAL AI + ML RESULTS SCREENSHOT

PASTE SCREENSHOT HERE

6. Conclusion

Week 5 strengthened practical understanding of end-to-end AI/ML workflows. The AI task covered preprocessing, training, prediction, evaluation, user interaction, and visualization. The ML task extended this work by comparing multiple models, tuning model hyperparameters, evaluating several metrics, and visualizing performance.

7. Submission Information

Intern	Saud Masood
Program	DevSphere Internship Program
Domain	Artificial Intelligence & Machine Learning
Week	05
Deadline	8 October 2026
Submission Email	devsphere1.info@gmail.com

Prepared for DevSphere Internship Program